

CMP 4040– spring 2026

ML

Report

Submitted to

Dr. Yahia Zakaria

Eng. Abdelrahman Kaseb

Submitted by

Name	Sec	ID	Contribution
Zeyad Alaa El Din	1	9220327	Phase 1 + Preprocessing
Seif Mohamed Ahmed Abdelbaky	1	9220386	Model Training + Hyper Parameter Tuning
Nour Khaled Ahmed	2	9220921	Model Training + Hyper Parameter Tuning
Kareem Wael Mohamed	2	9220602	Phase 1 + Preprocessing

Uber & Lyft Ride Price Prediction

Boston, MA — Ride-Hailing Price Forecasting

Problem Definition

Ride price prediction is a well-studied regression problem in transportation analytics, where the goal is to predict the fare of a ride given factors such as trip distance, time of day, weather conditions, and surge multipliers. The task is inherently challenging due to the dynamic nature of ride-hailing pricing, where fares fluctuate significantly based on demand and external conditions. The dataset covers real Uber and Lyft trips collected in Boston, MA, making it large enough to train robust models.

Evaluation Metrics	RMSE, R2 Score
Dataset Size	~693,000 rows, 57 features

Dataset Link: <https://www.kaggle.com/datasets/brllrb/uber-and-lyft-dataset-boston-ma>

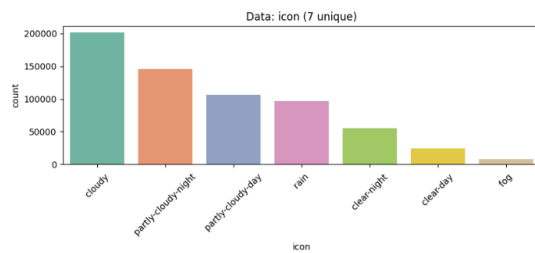
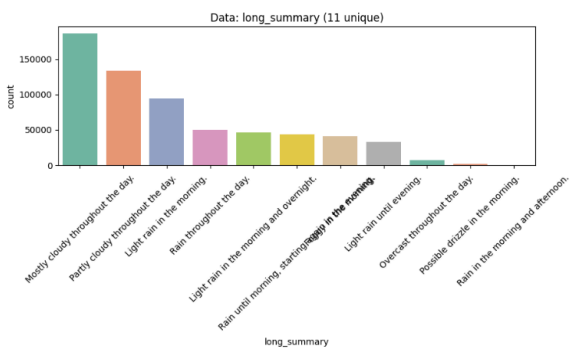
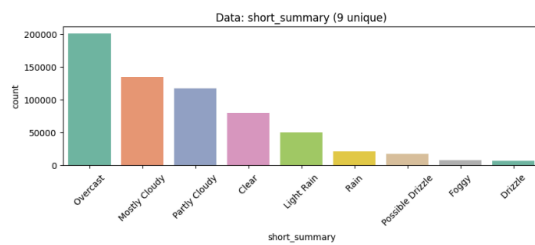
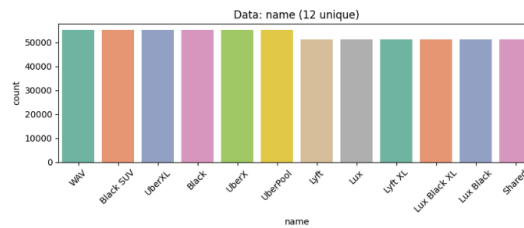
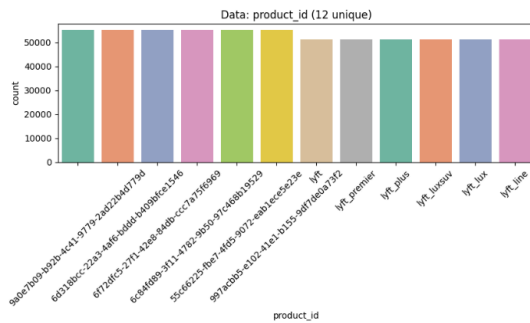
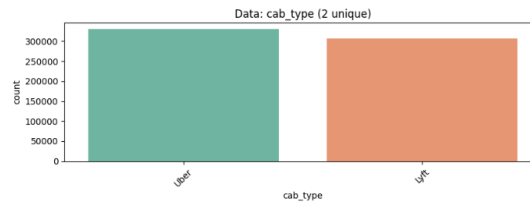
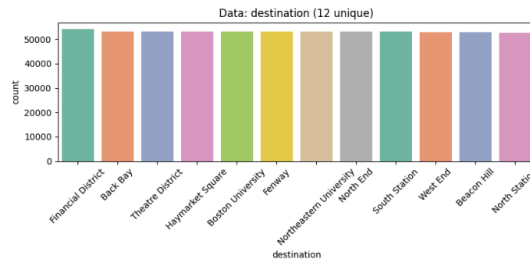
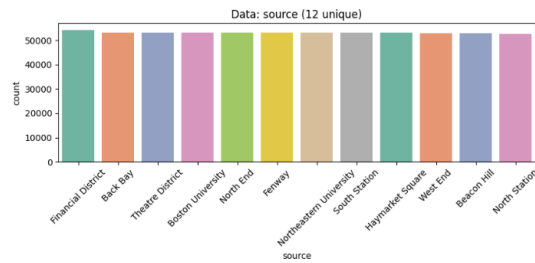
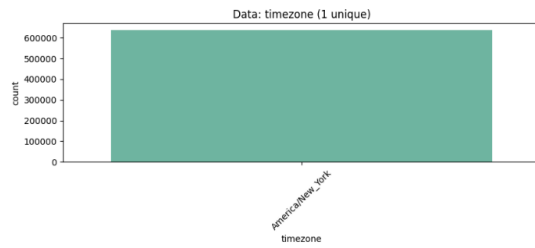
Dataset Analysis Results

1) Validation Results

- The dataset contains no duplicate records or formatting errors
- All time components (hour, day, month) are consistent, and all geographical coordinates (latitude/longitude) are valid.
- There are **55,095 missing values** in the price column
- Ride metrics are valid (some validation examples: no instances of zero-distance trips, identical source/destination pairs, or invalid surge multipliers).
- All weather and astronomical data passed validation, meaning temperatures, humidity levels, and solar cycles (sunrise/sunset) are valid.

Rows with missing prices were dropped.

2) Categorical Data Analysis

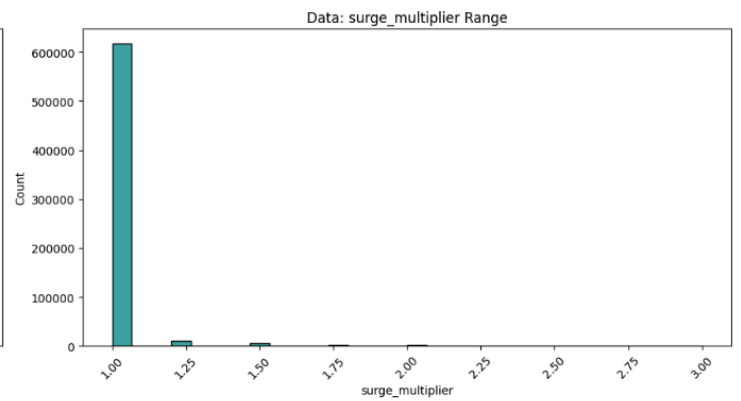
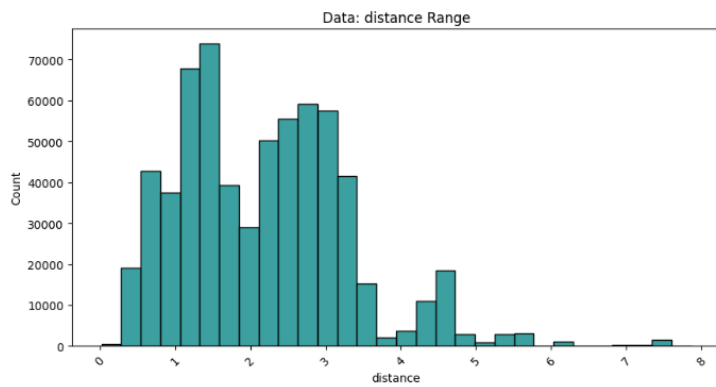
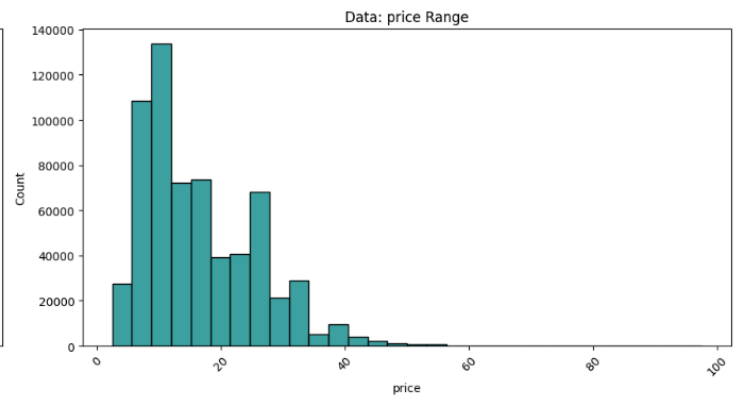
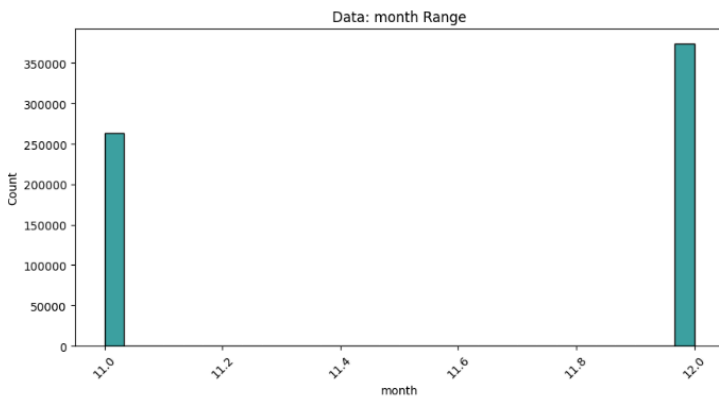
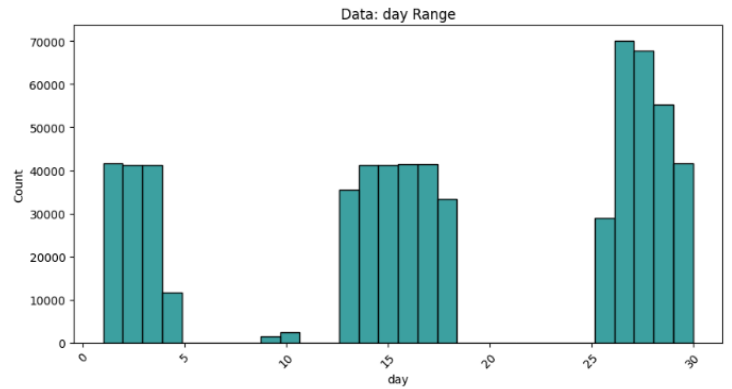
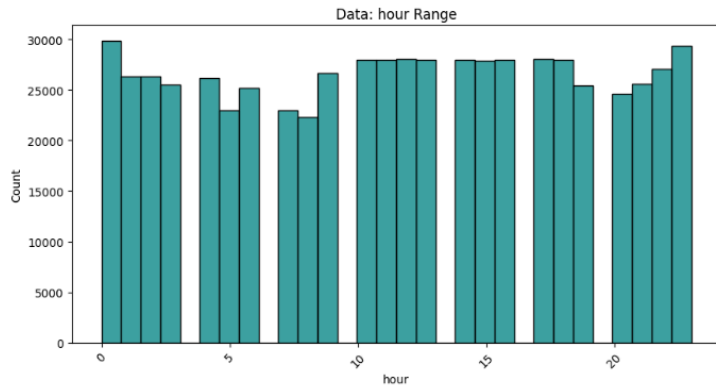


The following columns were dropped as they provide no extra information

['timezone', 'cab_type', 'product_id', 'short_summary', 'icon']

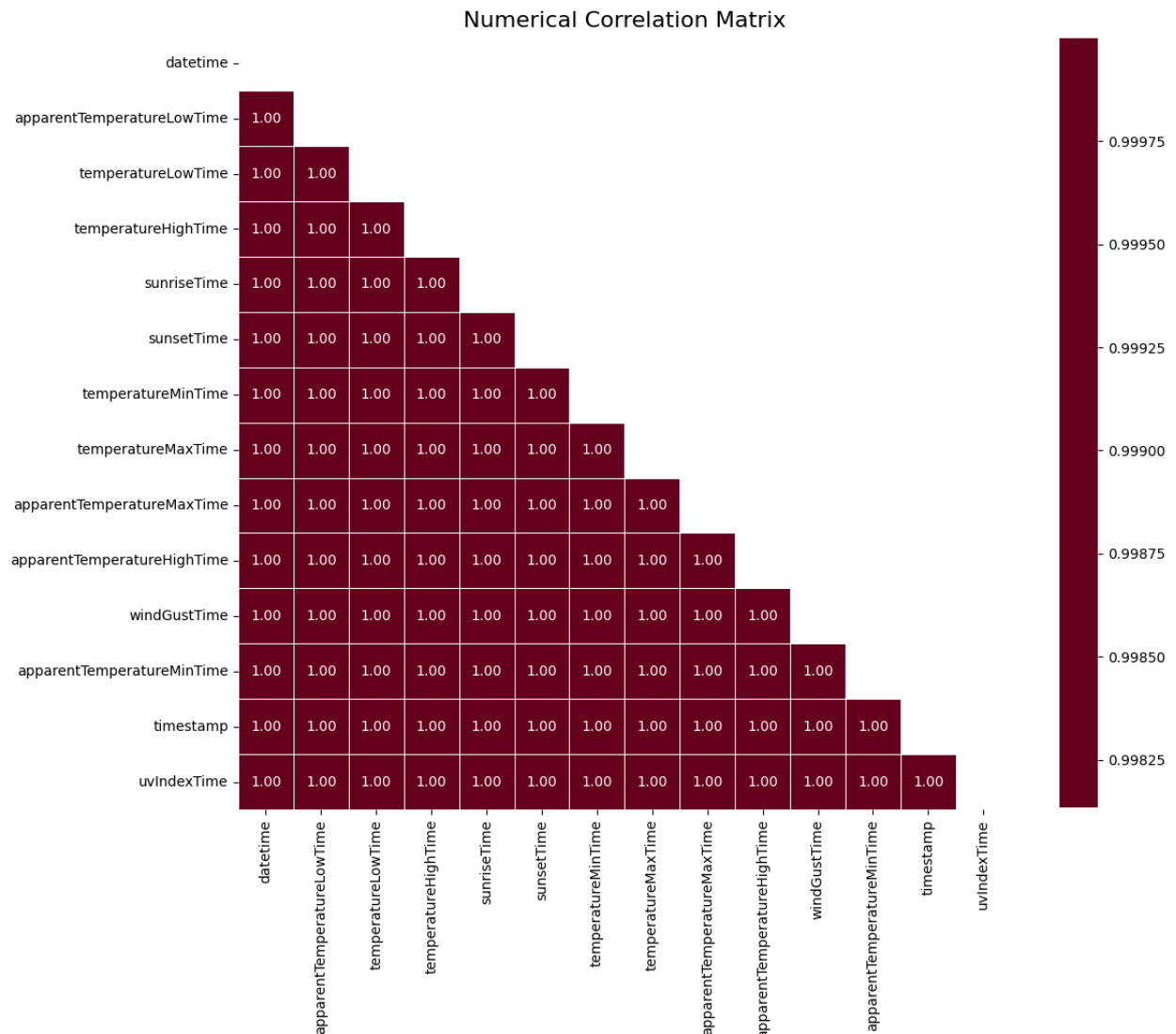
3) Numerical Data Analysis

Subset of histograms:



i) Highly correlated (Pearson correlation) groups of features were detected and dropped

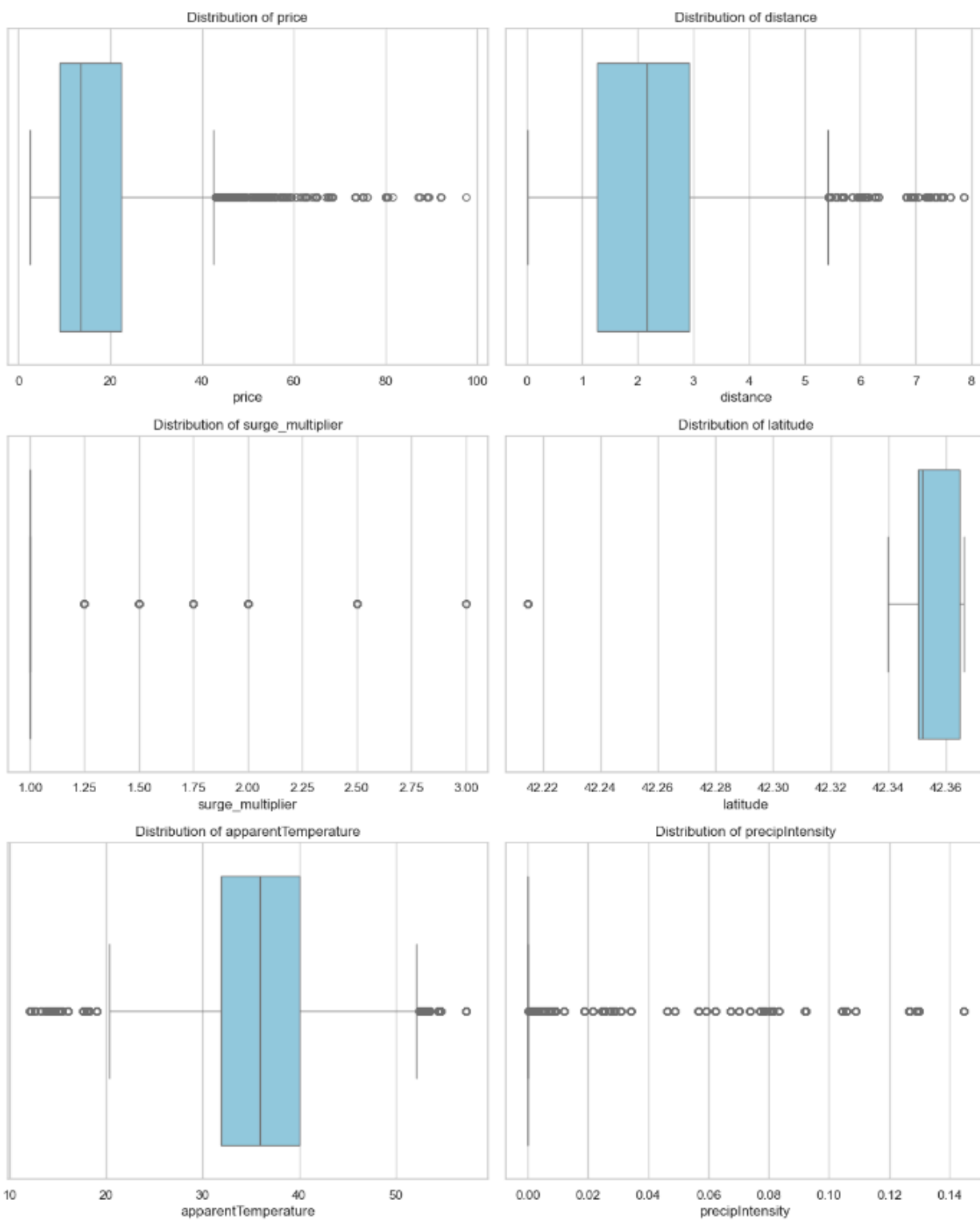
Example:



This resulted in **21 columns being dropped**

ii) Features with outliers were detected using IQR, however most of these were due to real extreme conditions which benefit model learning

Box plots were plotted to visualize them



4) Preprocessing

- Useless columns like ID were dropped
- A **Robust Scaler** is applied to numerical feature to scale features according to IQR and median to be resilient to outliers
- **OneHotEncoding** is used on categorical features
- Data is split into training and testing

Experimental Results

1) Baseline (ZeroR)

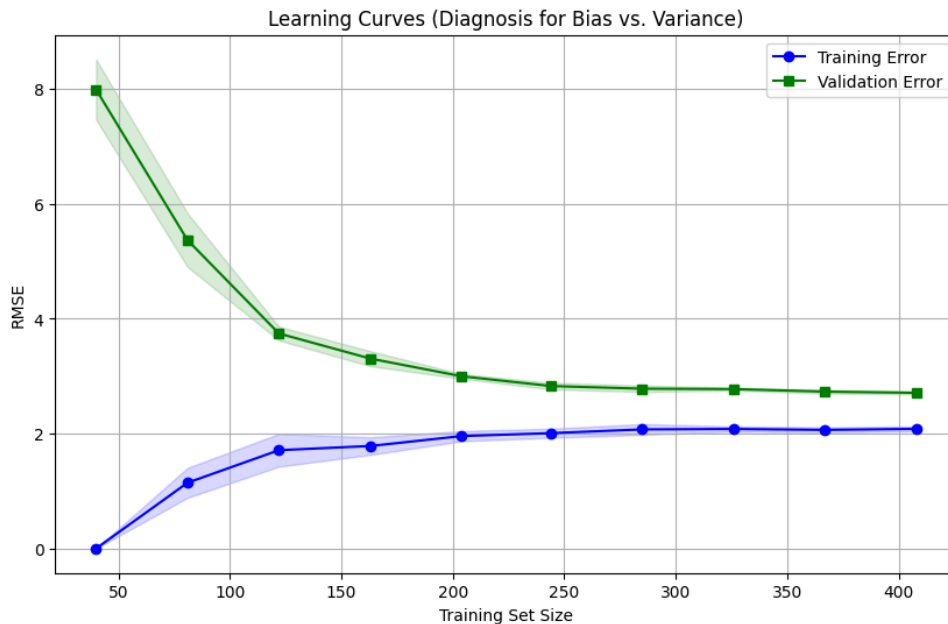
The mean of the prices in the training set is always used as the predicted price

ZeroR Baseline Results:

Mean Price (prediction): \$16.55

RMSE: \$9.34

2) Linear Regression



The Curve shows that the model has high bias (way too simple) and after using only 400 data points it converges

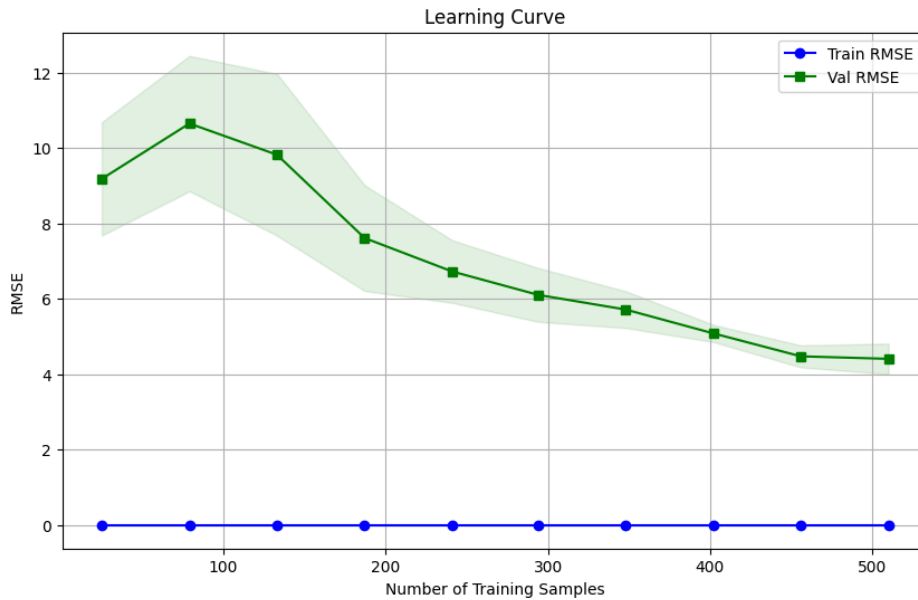
Conclusion: Increase model complexity

Training RMSE: 2.49

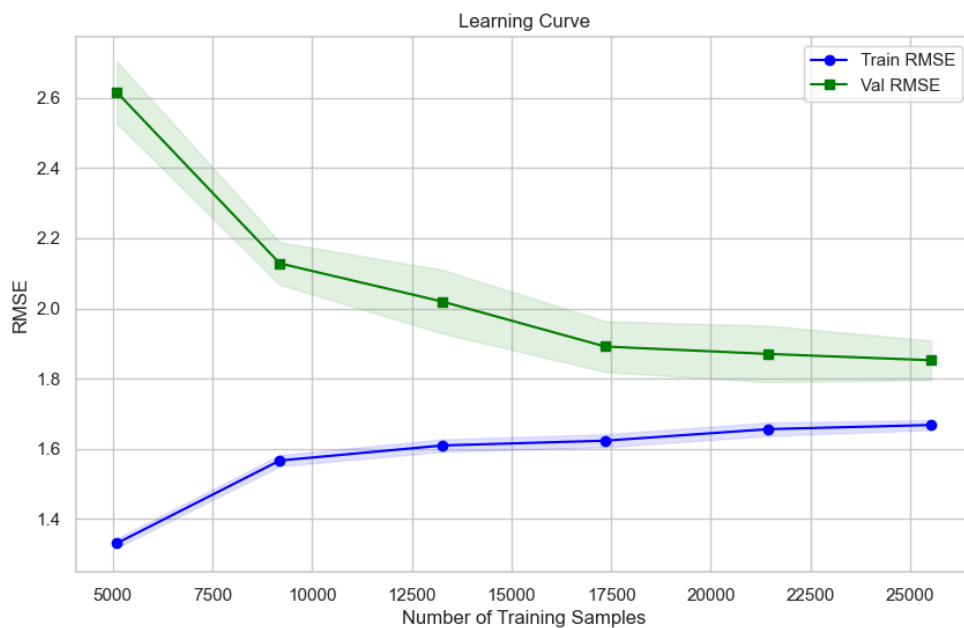
Validation RMSE: 2.49 (+/- 0.02)

3) Linear Regression with polynomial features (degree 2)

From (0 - 500) samples there's clear overfitting as the model tries to learn complex Relationships



From (5k - 25k) samples the model converges and it generalizes well to unseen data as It has learned the complex relationship after seeing many examples

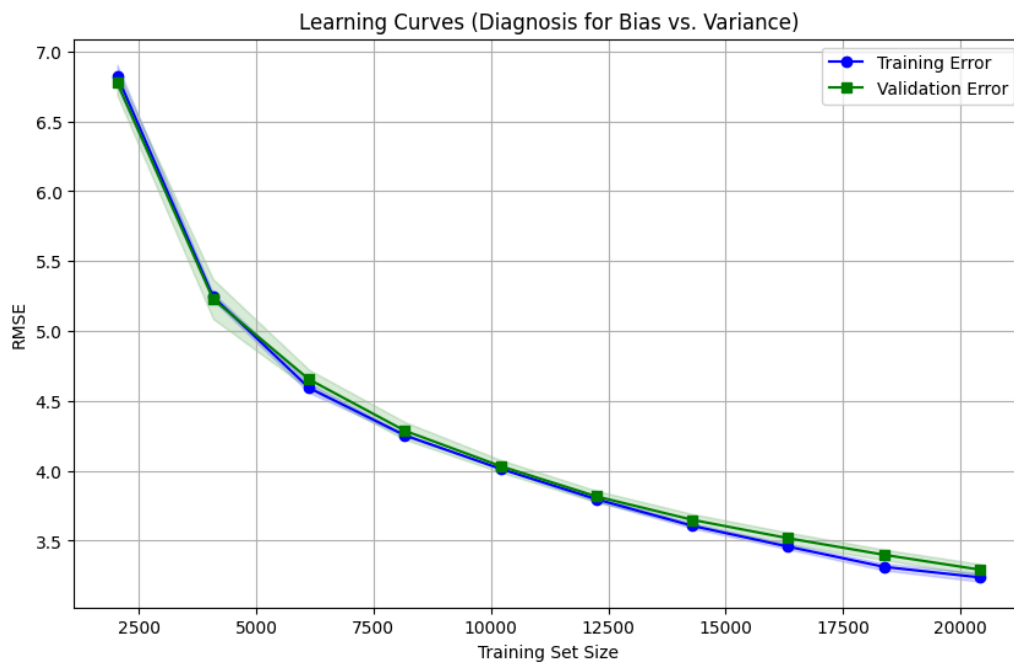


Training RMSE: 1.64

Validation RMSE: 1.87 (+/- 0.07)

4) SVM (Kernel: RBF)

i) $C = 1$



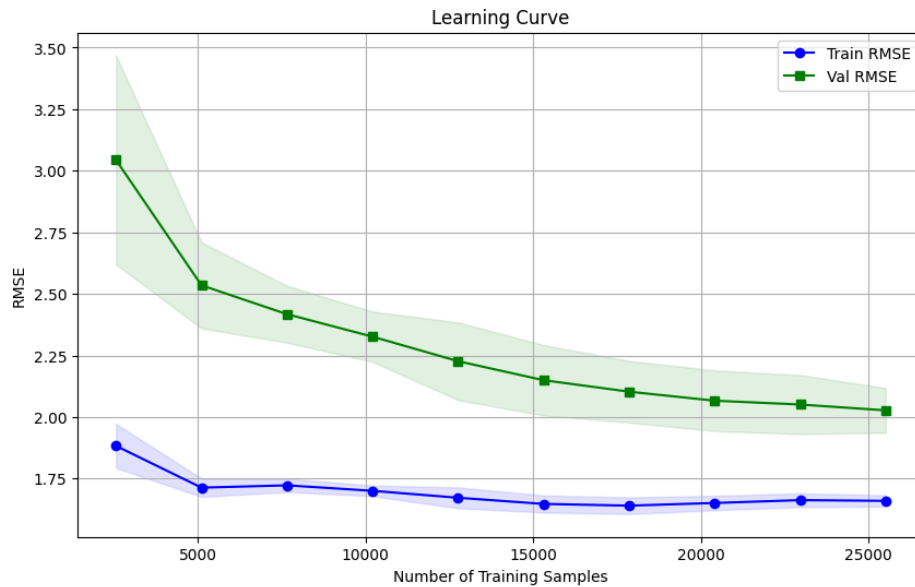
The model is too simple (High Bias)

We can increase complexity by increasing C (think of it as an inverse regularization term)

Training RMSE: 2.73

Validation RMSE: 2.77 (+/- 0.06)

ii) C = 50



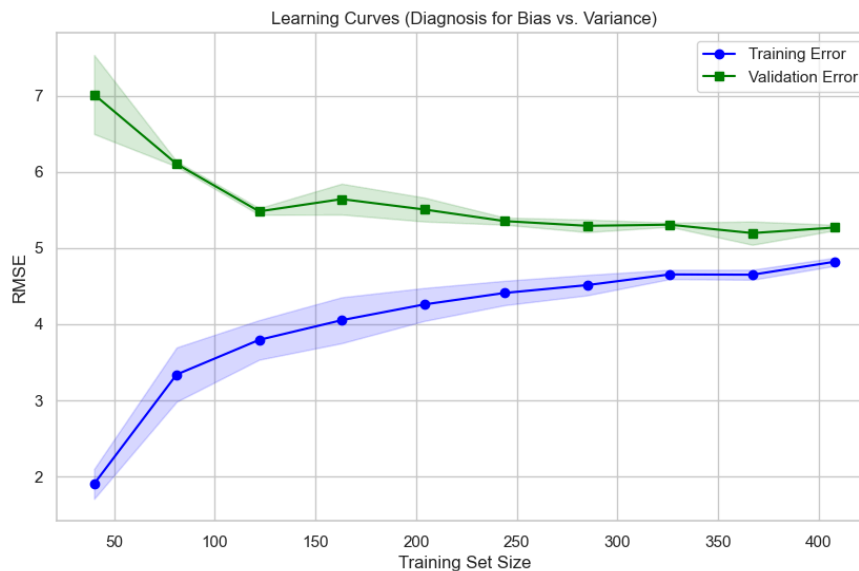
Any more complex than this and it would overfit too much and take too long to train

Training RMSE: 1.65

Validation RMSE: 2.07 (+/- 0.12)

5) Adaboost (Decision Tree Regressor)

i) n_estimators = 100, max_depth = 3

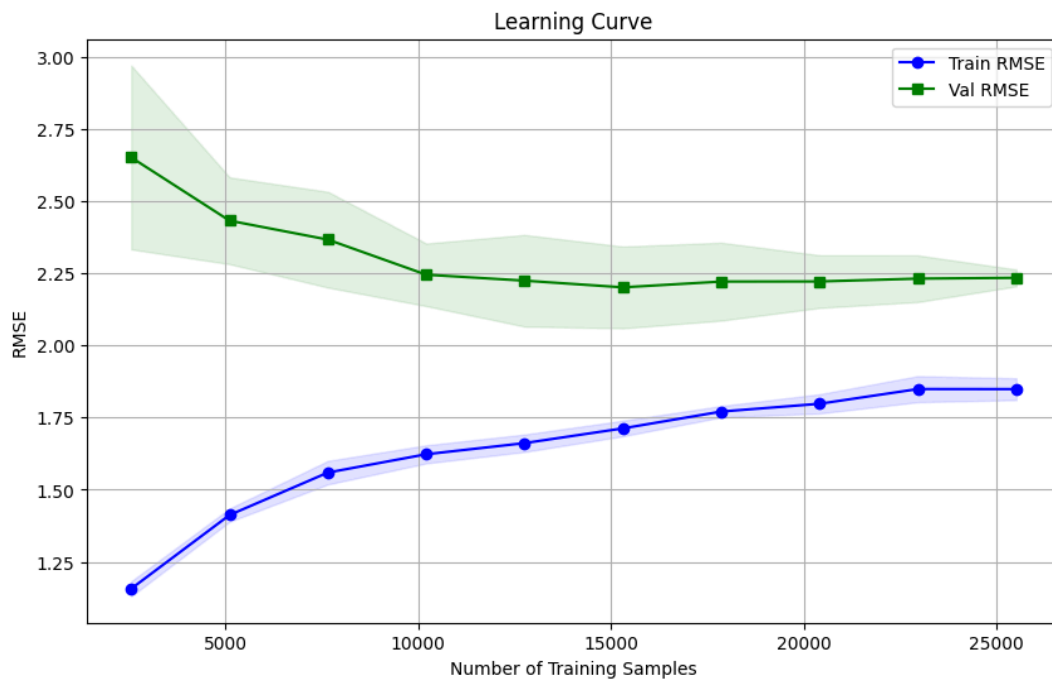


Too simple (High bias) - Need to increase complexity

Training RMSE: 4.72

Validation RMSE: 5.13 (+/- 0.50)

ii) $n_estimators = 200$, $max_depth = 10$



Training RMSE: 1.88

Validation RMSE: 2.14 (+/- 0.03)

Grid-based hyperparameter search was used to further tune hyperparameters and reach ($n_estimators = 100$, $max_depth = 15$, $learning_rate = 0.01$)

Tuned AdaBoost Results:

RMSE: \$1.91

Evaluation on Test Set:

Model	RMSE (\$)	R2
ZeroR	9.34	0
LinearRegression	2.49	0.929
LinearRegression (Polynomial Features)	1.82	0.962
SVM (RBF Kernel)	2.00	0.954
AdaBoost	1.83	0.962

Conclusion

The results show that underfitting was our biggest problem. Most models were too simple to learn the complex relationships in the data. To fix this, we always had to make our models more complex (like adding polynomial features or increasing the depth of our trees) to get better results. While Polynomial Linear Regression and AdaBoost performed best, the overall trend shows that increasing model complexity is the key to better accuracy. Moving forward, we will focus on using even more complex models and finer tuning to improve our predictions.